

Week 3 Task

SQL Analysis of Sales Data using Subqueries, CTEs, and Window Functions

Customer Sales Insights Mini-Project — Superstore Dataset

Celebal Technologies — Data Engineering Internship

Tool Used: SQL Server Management Studio (SSMS)

Dataset Source: Kaggle — Superstore Dataset

1. Objective

This assignment focuses on analyzing the Superstore sales dataset using SQL. The main objective is to practice Subqueries, Common Table Expressions (CTEs), and Window Functions to solve business-related questions such as identifying top customers, analyzing customer sales, and generating useful insights.

This task specifically covers:

- Loading the raw dataset into a staging table (superstore_raw)
- Designing normalized tables (customers, products, orders) from the staging table
- Populating the normalized tables using SELECT DISTINCT
- Using subqueries (simple and correlated) to filter data
- Using CTEs to simplify multi-step aggregations
- Using window functions (ROW_NUMBER, RANK, DENSE_RANK, NTILE) for ranking and segmentation
- Combining JOINS + CTEs + Window Functions to produce a final customer ranking report
- Answering specific business questions with the techniques above

2. Dataset Description

The Superstore dataset (from Kaggle) contains 9,994 order-line records from a US retail chain, spanning 2014–2017. Each row represents one product line within a customer order and includes:

- Order details: Row_ID, Order_ID, Order_Date, Ship_Date, Ship_Mode
- Customer details: Customer_ID, Customer_Name, Segment, Country
- Location details: City, State, Postal Code, Region
- Product details: Product_ID, Category, Sub-Category, Product_Name
- Metrics: Sales, Quantity, Discount, Profit

Because a single customer appears on many order rows (once per product purchased), the raw data needed to be split into separate customers, products, and orders tables before it could be analyzed relationally — To reduce data redundancy and make querying easier, the dataset was separated into customer, product, and order tables.

3. Tools and Environment

- Database engine: Microsoft SQL Server, managed through SSMS (SQL Server Management Studio)
- Data source: Superstore.csv, imported using the SSMS 'Import Flat File' wizard
- Concepts practiced: Subqueries (simple & correlated), CTEs, Window Functions (ROW_NUMBER, RANK, DENSE_RANK, NTILE), JOINS

STEP 1 : SETUP DATA

4. Step-by-Step: Loading Data and Building the Schema

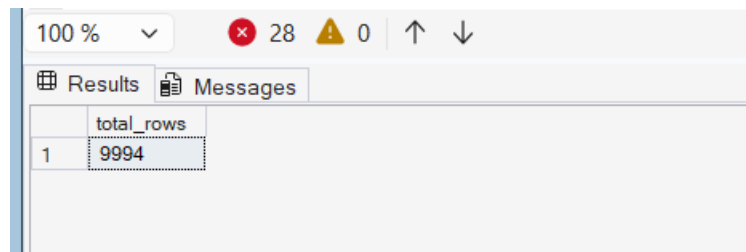
Step 4.1 — Import the CSV into a staging table (superstore_raw)

In SSMS: right-click the database → Tasks → Import Flat File → browse to Superstore.csv → name the destination table superstore_raw → let SSMS auto-detect data types on the preview screen → Finish.

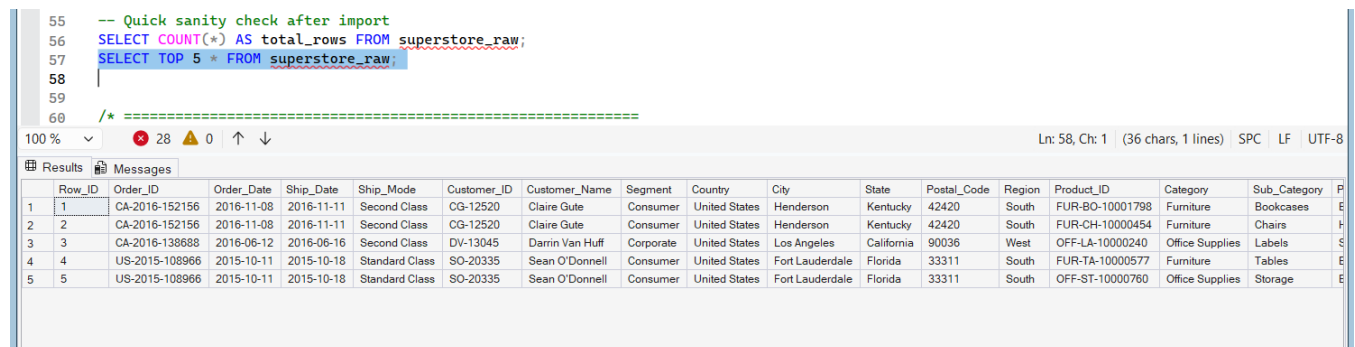
```
CREATE DATABASE SuperstoreDB;
USE SuperstoreDB;

-- After import, verify:
SELECT COUNT(*) AS total_rows FROM superstore_raw;
SELECT TOP 5 * FROM superstore_raw;
```

Actual result obtained: 9,994 rows loaded into superstore_raw .



	total_rows
1	9994



```
55 -- Quick sanity check after import
56 SELECT COUNT(*) AS total_rows FROM superstore_raw;
57 SELECT TOP 5 * FROM superstore_raw;
58
59
60 /* =====
```

Row_ID	Order_ID	Order_Date	Ship_Date	Ship_Mode	Customer_ID	Customer_Name	Segment	Country	City	State	Postal_Code	Region	Product_ID	Category	Sub_Category	P
1	CA-2016-152156	2016-11-08	2016-11-11	Second Class	CG-12520	Claire Gule	Consumer	United States	Henderson	Kentucky	42420	South	FUR-BO-10001798	Furniture	Bookcases	E
2	CA-2016-152156	2016-11-08	2016-11-11	Second Class	CG-12520	Claire Gule	Consumer	United States	Henderson	Kentucky	42420	South	FUR-CH-10000454	Furniture	Chairs	H
3	CA-2016-138688	2016-06-12	2016-06-16	Second Class	DV-13045	Darrin Van Huff	Corporate	United States	Los Angeles	California	90036	West	OFF-LA-10000240	Office Supplies	Labels	S
4	US-2015-108966	2015-10-11	2015-10-18	Standard Class	SO-20335	Sean O'Donnell	Consumer	United States	Fort Lauderdale	Florida	33311	South	FUR-TA-10000577	Furniture	Tables	E
5	US-2015-108966	2015-10-11	2015-10-18	Standard Class	SO-20335	Sean O'Donnell	Consumer	United States	Fort Lauderdale	Florida	33311	South	OFF-ST-10000760	Office Supplies	Storage	E

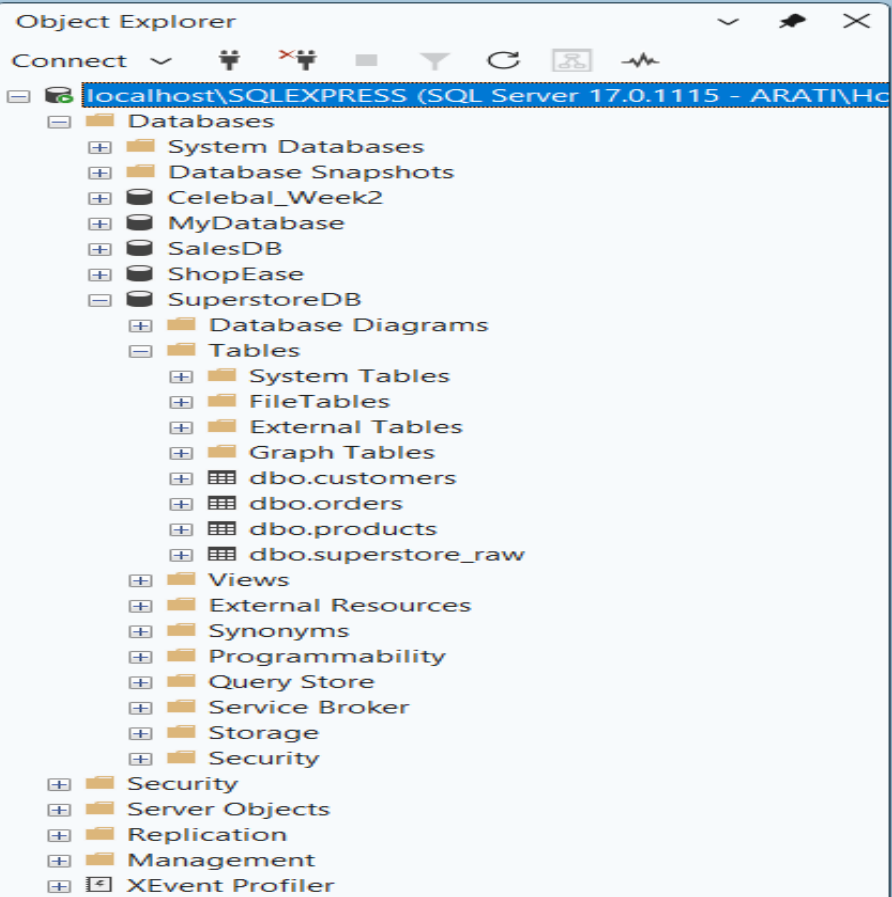
Step 4.2 — Create the normalized tables

Three tables were created to model the data properly: customers, products, and orders.

```
CREATE TABLE customers (
    customer_id VARCHAR(20) PRIMARY KEY,
    customer_name VARCHAR(100),
    segment VARCHAR(30),
    country VARCHAR(50)
);

CREATE TABLE products (
    product_id VARCHAR(30) PRIMARY KEY,
    category VARCHAR(30),
    sub_category VARCHAR(30),
    product_name VARCHAR(255)
);
```

```
CREATE TABLE orders (  
    row_id          INT PRIMARY KEY,  
    order_id        VARCHAR(20),  
    order_date      DATE,  
    ship_date       DATE,  
    ship_mode       VARCHAR(30),  
    customer_id     VARCHAR(20),  
    product_id      VARCHAR(30),  
    city            VARCHAR(50),  
    state           VARCHAR(50),  
    postal_code     VARCHAR(10),  
    region          VARCHAR(20),  
    sales           DECIMAL(10,2),  
    quantity        INT,  
    discount        DECIMAL(4,2),  
    profit          DECIMAL(10,2)  
);
```



Step 4.3 — Populate the tables using SELECT DISTINCT

```
INSERT INTO customers (customer_id, customer_name, segment, country)
SELECT DISTINCT Customer_ID, Customer_Name, Segment, Country
FROM superstore_raw;

-- A handful of Product IDs repeat with a different product name in this
-- dataset (a known data-quality quirk), so ROW_NUMBER() is used to keep
-- one canonical name per ID instead of a plain SELECT DISTINCT.
INSERT INTO products (product_id, category, sub_category, product_name)
SELECT product_id, category, sub_category, product_name
FROM (
    SELECT ProductID AS product_id, Category AS category,
           SubCategory AS sub_category, ProductName AS product_name,
           ROW_NUMBER() OVER (PARTITION BY ProductID ORDER BY ProductName) AS rn
    FROM superstore_raw
) t
WHERE rn = 1;

INSERT INTO orders
SELECT Row_ID, Order_ID, Order_Date, Ship_Date, Ship_Mode, Customer_ID, Product_ID,
       City, State, Postal_Code, Region, Sales, Quantity, Discount, Profit
FROM superstore_raw;
```

Row counts after population:

```
126 -- Row-count check
127 SELECT 'customers' AS table_name, COUNT(*) AS row_count FROM customers
128 UNION ALL
129 SELECT 'products', COUNT(*) FROM products
130 UNION ALL
131 SELECT 'orders', COUNT(*) FROM orders;
132
```

100 % 28 0 ↑ ↓

Results Messages

	table_name	row_count
1	customers	793
2	products	1862
3	orders	9994

STEP 2 : Performed Required Queries

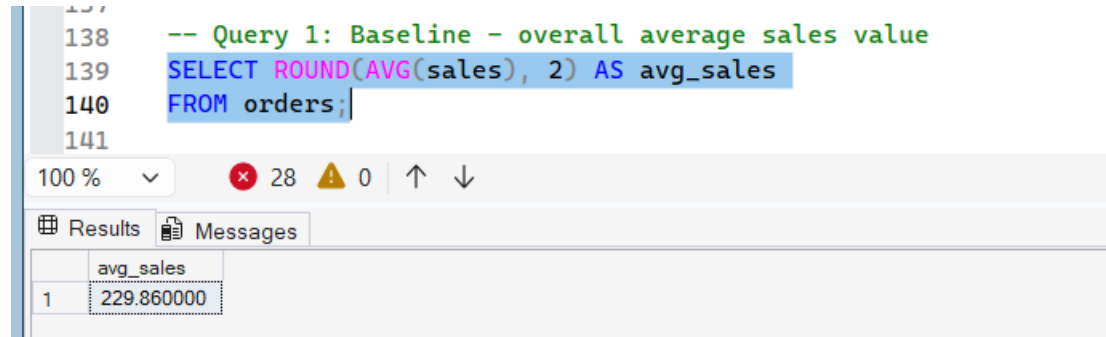
5. Subqueries

Query 1: Overall average sales value (baseline)

SQL Query:

```
SELECT ROUND(AVG(sales), 2) AS avg_sales  
FROM orders;
```

Output :



The screenshot shows a SQL IDE interface. The top pane displays the SQL query: `-- Query 1: Baseline - overall average sales value`, `SELECT ROUND(AVG(sales), 2) AS avg_sales`, and `FROM orders;`. The bottom pane shows the results of the query in a table with one column, `avg_sales`, and one row with the value `229.860000`. The interface includes a toolbar with a zoom level of 100%, a status bar showing 28 errors and 0 warnings, and tabs for Results and Messages.

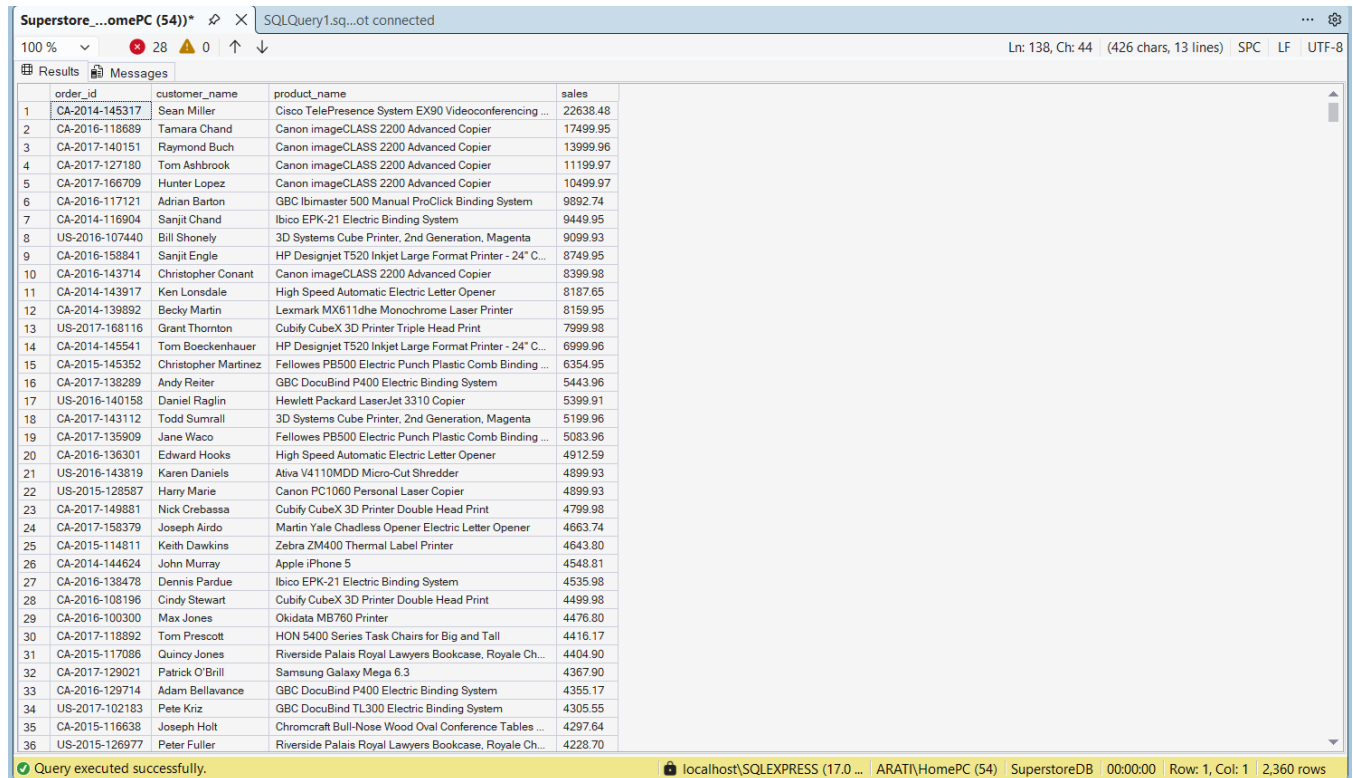
	avg_sales
1	229.860000

1. Orders where sales are greater than average sales .

SQL Query:

```
SELECT o.order_id, c.customer_name, p.product_name, o.sales
FROM orders o
JOIN customers c ON o.customer_id = c.customer_id
JOIN products p ON o.product_id = p.product_id
WHERE o.sales > (SELECT AVG(sales) FROM orders)
ORDER BY o.sales DESC;
```

Output :



	order_id	customer_name	product_name	sales
1	CA-2014-145317	Sean Miller	Cisco TelePresence System EX90 Videoconferencing ...	22638.48
2	CA-2016-118689	Tamara Chand	Canon imageCLASS 2200 Advanced Copier	17499.95
3	CA-2017-140151	Raymond Buch	Canon imageCLASS 2200 Advanced Copier	13999.96
4	CA-2017-127180	Tom Ashbrook	Canon imageCLASS 2200 Advanced Copier	11199.97
5	CA-2017-166709	Hunter Lopez	Canon imageCLASS 2200 Advanced Copier	10499.97
6	CA-2016-117121	Adrian Barton	GBC Ibimaster 500 Manual ProClick Binding System	9892.74
7	CA-2014-116904	Sanjit Chand	Ibico EPK-21 Electric Binding System	9449.95
8	US-2016-107440	Bill Shonely	3D Systems Cube Printer, 2nd Generation, Magenta	9099.93
9	CA-2016-158841	Sanjit Engle	HP Designjet T520 Inkjet Large Format Printer - 24" C...	8749.95
10	CA-2016-143714	Christopher Conant	Canon imageCLASS 2200 Advanced Copier	8399.98
11	CA-2014-143917	Ken Lonsdale	High Speed Automatic Electric Letter Opener	8187.65
12	CA-2014-139892	Becky Martin	Lexmark MX611dhe Monochrome Laser Printer	8159.95
13	US-2017-168116	Grant Thornton	Cubify CubeX 3D Printer Triple Head Print	7999.98
14	CA-2014-145541	Tom Boeckenhauer	HP Designjet T520 Inkjet Large Format Printer - 24" C...	6999.96
15	CA-2015-145352	Christopher Martinez	Fellowes PB500 Electric Punch Plastic Comb Binding ...	6354.95
16	CA-2017-138289	Andy Reiter	GBC DocuBind P400 Electric Binding System	5443.96
17	US-2016-140158	Daniel Raglin	Hewlett Packard LaserJet 3310 Copier	5399.91
18	CA-2017-143112	Todd Sumrall	3D Systems Cube Printer, 2nd Generation, Magenta	5199.96
19	CA-2017-135909	Jane Waco	Fellowes PB500 Electric Punch Plastic Comb Binding ...	5083.96
20	CA-2016-136301	Edward Hooks	High Speed Automatic Electric Letter Opener	4912.59
21	US-2016-143819	Karen Daniels	Ativa V4110MDD Micro-Cut Shredder	4899.93
22	US-2015-128587	Harry Marie	Canon PC1060 Personal Laser Copier	4899.93
23	CA-2017-149881	Nick Crebassa	Cubify CubeX 3D Printer Double Head Print	4799.98
24	CA-2017-158379	Joseph Airdo	Martin Yale Chadless Opener Electric Letter Opener	4663.74
25	CA-2015-114811	Keith Dawkins	Zebra ZM400 Thermal Label Printer	4643.80
26	CA-2014-144624	John Murray	Apple iPhone 5	4548.81
27	CA-2016-138478	Dennis Pardue	Ibico EPK-21 Electric Binding System	4535.98
28	CA-2016-108196	Cindy Stewart	Cubify CubeX 3D Printer Double Head Print	4499.98
29	CA-2016-100300	Max Jones	Okidata MB760 Printer	4476.80
30	CA-2017-118892	Tom Prescott	HON 5400 Series Task Chairs for Big and Tall	4416.17
31	CA-2015-117086	Quincy Jones	Riverside Palais Royal Lawyers Bookcase, Royale Ch...	4404.90
32	CA-2017-129021	Patrick O'Brill	Samsung Galaxy Mega 6.3	4367.90
33	CA-2016-129714	Adam Bellavance	GBC DocuBind P400 Electric Binding System	4355.17
34	US-2017-102183	Pete Kriz	GBC DocuBind TL300 Electric Binding System	4305.55
35	CA-2015-116638	Joseph Holt	Chromcraft Bull-Nose Wood Oval Conference Tables ...	4297.64
36	US-2015-126977	Peter Fuller	Riverside Palais Royal Lawyers Bookcase, Royale Ch...	4228.70

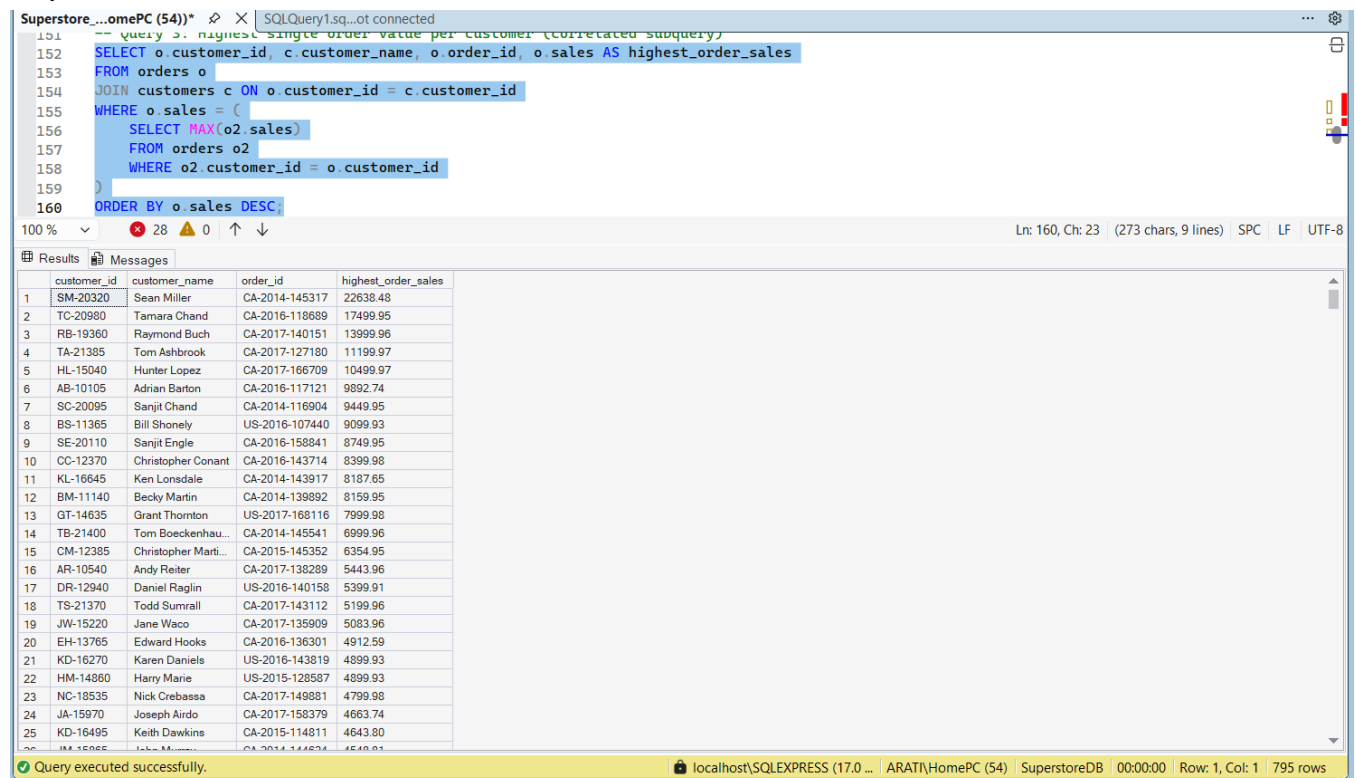
Query executed successfully. localhost\SQLEXPRESS (17.0 ... ARATI\HomePC (54) SuperstoreDB 00:00:00 Row: 1, Col: 1 2,360 rows

2. Highest sales order for each customer

SQL Query:

```
SELECT o.customer_id, c.customer_name, o.order_id, o.sales AS highest_order_sales
FROM orders o
JOIN customers c ON o.customer_id = c.customer_id
WHERE o.sales = (
    SELECT MAX(o2.sales)
    FROM orders o2
    WHERE o2.customer_id = o.customer_id
)
ORDER BY o.sales DESC;
```

Output :



The screenshot shows a SQL Server Enterprise Manager window with a query editor and a results pane. The query editor contains the SQL query for finding the highest sales order for each customer. The results pane displays a table with 25 rows of data, sorted by sales amount in descending order. The status bar at the bottom indicates that the query was executed successfully and returned 795 rows.

	customer_id	customer_name	order_id	highest_order_sales
1	SM-20320	Sean Miller	CA-2014-145317	22638.48
2	TC-20980	Tamara Chand	CA-2016-118689	17499.95
3	RB-19360	Raymond Buch	CA-2017-140151	13999.96
4	TA-21385	Tom Ashbrook	CA-2017-127180	11199.97
5	HL-15040	Hunter Lopez	CA-2017-166709	10499.97
6	AB-10105	Adrian Barton	CA-2016-117121	9892.74
7	SC-20095	Sanjit Chand	CA-2014-116904	9449.95
8	BS-11365	Bill Shonely	US-2016-107440	9099.93
9	SE-20110	Sanjit Engle	CA-2016-158841	8749.95
10	CC-12370	Christopher Conant	CA-2016-143714	8399.98
11	KL-16645	Ken Lonsdale	CA-2014-143917	8187.65
12	BM-11140	Becky Martin	CA-2014-139892	8159.95
13	GT-14635	Grant Thornton	US-2017-168116	7999.98
14	TB-21400	Tom Boeckenhau...	CA-2014-145541	6999.96
15	CM-12385	Christopher Marti...	CA-2015-145352	6354.95
16	AR-10540	Andy Reiter	CA-2017-138289	5443.96
17	DR-12940	Daniel Raglin	US-2016-140158	5399.91
18	TS-21370	Todd Sumrall	CA-2017-143112	5199.96
19	JW-15220	Jane Waco	CA-2017-135909	5083.96
20	EH-13765	Edward Hooks	CA-2016-136301	4912.59
21	KD-16270	Karen Daniels	US-2016-143819	4899.93
22	HM-14860	Harry Marie	US-2015-128587	4899.93
23	NC-18535	Nick Crebassa	CA-2017-149881	4799.98
24	JA-15970	Joseph Airdo	CA-2017-158379	4663.74
25	KD-16495	Keith Dawkins	CA-2015-114811	4643.80

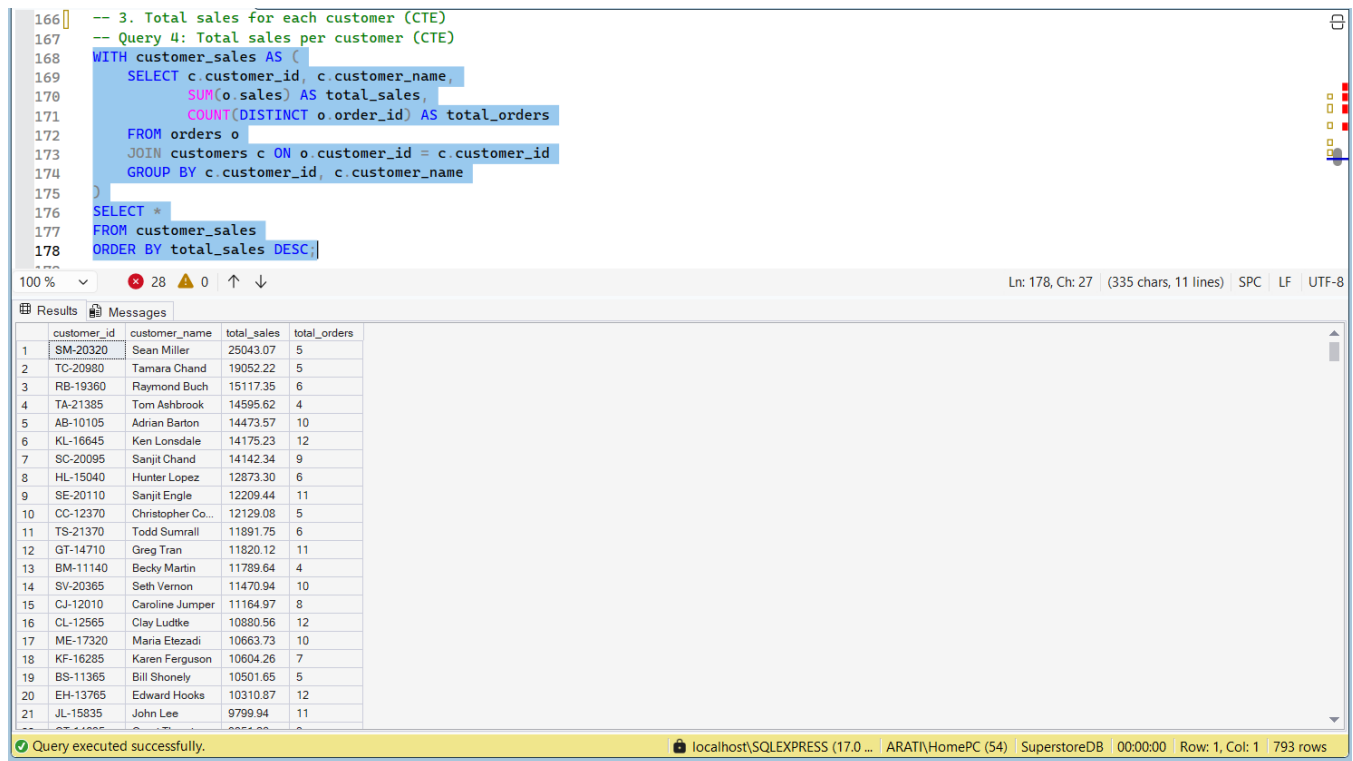
6. Common Table Expressions (CTEs)

3. Total sales for each customer (CTE)

SQL Query:

```
WITH customer_sales AS (  
    SELECT c.customer_id, c.customer_name,  
           SUM(o.sales) AS total_sales,  
           COUNT(DISTINCT o.order_id) AS total_orders  
    FROM orders o  
    JOIN customers c ON o.customer_id = c.customer_id  
    GROUP BY c.customer_id, c.customer_name  
)  
SELECT *  
FROM customer_sales  
ORDER BY total_sales DESC;
```

Output :



The screenshot shows a SQL query window with the following code:

```
-- 3. Total sales for each customer (CTE)  
-- Query 4: Total sales per customer (CTE)  
WITH customer_sales AS (  
    SELECT c.customer_id, c.customer_name,  
           SUM(o.sales) AS total_sales,  
           COUNT(DISTINCT o.order_id) AS total_orders  
    FROM orders o  
    JOIN customers c ON o.customer_id = c.customer_id  
    GROUP BY c.customer_id, c.customer_name  
)  
SELECT *  
FROM customer_sales  
ORDER BY total_sales DESC;
```

The output window displays the results of the query, showing 21 rows of data. The columns are customer_id, customer_name, total_sales, and total_orders. The data is sorted by total_sales in descending order.

	customer_id	customer_name	total_sales	total_orders
1	SM-20320	Sean Miller	25043.07	5
2	TC-20980	Tamara Chand	19052.22	5
3	RB-19360	Raymond Buch	15117.35	6
4	TA-21385	Tom Ashbrook	14595.62	4
5	AB-10105	Adrian Barton	14473.57	10
6	KL-16645	Ken Lonsdale	14175.23	12
7	SC-20095	Sanjit Chand	14142.34	9
8	HL-15040	Hunter Lopez	12873.30	6
9	SE-20110	Sanjit Engle	12209.44	11
10	CC-12370	Christopher Co...	12129.08	5
11	TS-21370	Todd Sumrall	11891.75	6
12	GT-14710	Greg Tran	11820.12	11
13	BM-11140	Becky Martin	11789.64	4
14	SV-20365	Seth Vernon	11470.94	10
15	CJ-12010	Caroline Jumper	11164.97	8
16	CL-12565	Clay Ludtke	10880.56	12
17	ME-17320	Maria Elezadi	10663.73	10
18	KF-16285	Karen Ferguson	10604.26	7
19	BS-11365	Bill Shonely	10501.65	5
20	EH-13765	Edward Hooks	10310.87	12
21	JL-15835	John Lee	9799.94	11

The status bar at the bottom indicates: Query executed successfully. localhost\SQLEXPRESS (17.0 ... ARATI\HomePC (54) SuperstoreDB 00:00:00 Row: 1, Col: 1 793 rows

4. Customers whose total sales are above average (CTE + Subquery)

SQL Query:

```
WITH customer_sales AS (  
    SELECT c.customer_id, c.customer_name, SUM(o.sales) AS total_sales  
    FROM orders o  
    JOIN customers c ON o.customer_id = c.customer_id  
    GROUP BY c.customer_id, c.customer_name  
)  
SELECT customer_id, customer_name, ROUND(total_sales, 2) AS total_sales  
FROM customer_sales  
WHERE total_sales > (SELECT AVG(total_sales) FROM customer_sales)  
ORDER BY total_sales DESC;
```

Output :

The screenshot shows a SQL Server Enterprise Manager window with a query editor and a results pane. The query editor contains the SQL code from the previous block. The results pane shows a table with 25 rows of data. The status bar at the bottom indicates that the query was executed successfully and returned 294 rows.

	customer_id	customer_name	total_sales
1	SM-20320	Sean Miller	25043.07
2	TC-20980	Tamara Chand	19052.22
3	RB-19360	Raymond Buch	15117.35
4	TA-21385	Tom Ashbrook	14595.62
5	AB-10105	Adrian Barton	14473.57
6	KL-16645	Ken Lonsdale	14175.23
7	SC-20095	Sanjit Chand	14142.34
8	HL-15040	Hunter Lopez	12873.30
9	SE-20110	Sanjit Engle	12209.44
10	CC-12370	Christopher Conant	12129.08
11	TS-21370	Todd Sumrall	11891.75
12	GT-14710	Greg Tran	11820.12
13	BM-11140	Becky Martin	11789.64
14	SV-20365	Seth Vernon	11470.94
15	CJ-12010	Caroline Jumper	11164.97
16	CL-12565	Clay Ludtke	10880.56
17	ME-17320	Maria Elezadi	10663.73
18	KF-16285	Karen Ferguson	10604.26
19	BS-11365	Bill Shonely	10501.65
20	EH-13765	Edward Hooks	10310.87
21	JL-15835	John Lee	9799.94
22	GT-14635	Grant Thornton	9351.20
23	HW-14935	Helen Wasserman	9300.25
24	TB-21400	Tom Boeckenhau...	9133.99
25	PF-19120	Peter Fuller	9062.86

Query executed successfully. | localhost\SQLEXPRESS (17.0 ... | ARATI\HomePC (54) | SuperstoreDB | 00:00:00 | Row: 1, Col: 1 | 294 rows

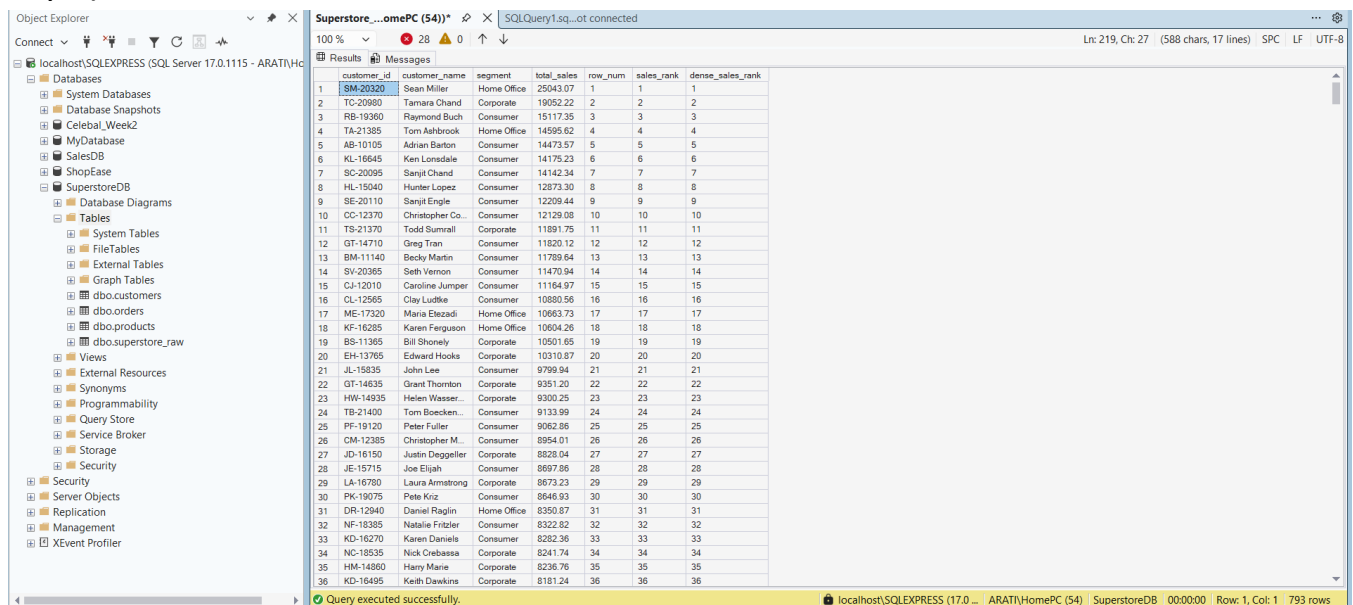
7. Window Functions

5. Rank all customers based on total sales (Window Function)

SQL Query:

```
WITH customer_sales AS (  
    SELECT c.customer_id, c.customer_name, c.segment,  
           SUM(o.sales) AS total_sales  
    FROM orders o  
    JOIN customers c ON o.customer_id = c.customer_id  
    GROUP BY c.customer_id, c.customer_name, c.segment  
)  
SELECT  
    customer_id,  
    customer_name,  
    segment,  
    ROUND(total_sales, 2) AS total_sales,  
    ROW_NUMBER() OVER (ORDER BY total_sales DESC) AS row_num,  
    RANK() OVER (ORDER BY total_sales DESC) AS sales_rank,  
    DENSE_RANK() OVER (ORDER BY total_sales DESC) AS dense_sales_rank  
FROM customer_sales  
ORDER BY total_sales DESC;
```

Output):



customer_id	customer_name	segment	total_sales	row_num	sales_rank	dense_sales_rank
SM-20320	Sean Miller	Home Office	25043.07	1	1	1
TC-20960	Tamara Chand	Corporate	19052.22	2	2	2
RB-10960	Raymond Bush	Consumer	15117.35	3	3	3
TA-21355	Tom Ashbrook	Home Office	14595.62	4	4	4
AB-10105	Adrian Barton	Consumer	14473.57	5	5	5
KL-16645	Ken Lonsdale	Consumer	14175.23	6	6	6
SC-20095	Sanjit Chand	Consumer	14142.34	7	7	7
HL-15040	Hunter Lopez	Consumer	12873.30	8	8	8
SE-20110	Sanjit Engle	Consumer	12209.44	9	9	9
CC-12370	Christopher Co...	Consumer	12129.98	10	10	10
TB-21370	Todd Sumail	Corporate	11891.75	11	11	11
GT-14710	Oreg Tran	Consumer	11820.12	12	12	12
BM-11140	Becky Martin	Consumer	11789.64	13	13	13
SV-20365	Seth Vernon	Consumer	11470.94	14	14	14
CJ-12010	Caroline Jumper	Consumer	11164.97	15	15	15
CL-12565	Clay Ludtke	Consumer	10880.56	16	16	16
ME-17320	Maria Etezadi	Home Office	10663.73	17	17	17
KF-16285	Karen Ferguson	Home Office	10604.26	18	18	18
BS-11365	Bill Shively	Corporate	10501.65	19	19	19
EH-13785	Edward Hooks	Corporate	10310.87	20	20	20
JL-15835	John Lee	Consumer	9799.94	21	21	21
GT-14635	Grant Thornton	Corporate	9351.20	22	22	22
HW-14935	Helen Wasser...	Corporate	9300.25	23	23	23
TB-21400	Tom Boecken...	Consumer	9133.99	24	24	24
PF-19120	Peter Fuller	Consumer	9062.86	25	25	25
CM-12385	Christopher M...	Consumer	8954.01	26	26	26
JD-16150	Justin Deggeller	Corporate	8828.04	27	27	27
JC-15715	Joe Digh	Consumer	8697.86	28	28	28
LA-16780	Laura Armstrong	Corporate	8673.23	29	29	29
PK-19075	Pete Kriz	Consumer	8646.93	30	30	30
DR-12940	Daniel Raglin	Home Office	8350.87	31	31	31
NF-18385	Natalie Fritzer	Consumer	8322.82	32	32	32
KD-16270	Karen Daniels	Consumer	8282.36	33	33	33
NC-18535	Nick Crebassa	Corporate	8241.74	34	34	34
NM-14860	Hany Marie	Corporate	8236.76	35	35	35
KD-16495	Keith Dawkins	Corporate	8181.24	36	36	36

6. Row numbers for each order within a customer (Window Function + PARTITION BY)

```
SELECT o.customer_id, c.customer_name, o.order_id, o.order_date, o.sales,  
       ROW_NUMBER() OVER (PARTITION BY o.customer_id ORDER BY o.order_date) AS order_row_num  
FROM orders o  
JOIN customers c ON o.customer_id = c.customer_id  
ORDER BY o.customer_id, order_row_num;
```

```
221 --6. Row numbers for each order within a customer (Window Function + PARTITION BY)
222 SELECT o.customer_id, c.customer_name, o.order_id, o.order_date, o.sales,
223        ROW_NUMBER() OVER (PARTITION BY o.customer_id ORDER BY o.order_date) AS order_row_num
224 FROM orders o
225 JOIN customers c ON o.customer_id = c.customer_id
226 ORDER BY o.customer_id, order_row_num
```

100 % 28 0

Ln: 222, Ch: 1 (269 chars, 5 lines) SPC LF UTF

	customer_id	customer_name	order_id	order_date	sales	order_row_num
1	AA-10315	Alex Avila	CA-2014-128055	2014-03-31	673.57	1
2	AA-10315	Alex Avila	CA-2014-128055	2014-03-31	52.98	2
3	AA-10315	Alex Avila	CA-2014-138100	2014-09-15	14.94	3
4	AA-10315	Alex Avila	CA-2014-138100	2014-09-15	14.56	4
5	AA-10315	Alex Avila	CA-2015-121391	2015-10-04	26.96	5
6	AA-10315	Alex Avila	CA-2016-103982	2016-03-03	3930.07	6
7	AA-10315	Alex Avila	CA-2016-103982	2016-03-03	2.30	7
8	AA-10315	Alex Avila	CA-2016-103982	2016-03-03	431.98	8
9	AA-10315	Alex Avila	CA-2016-103982	2016-03-03	41.72	9
10	AA-10315	Alex Avila	CA-2017-147039	2017-06-29	362.94	10
11	AA-10315	Alex Avila	CA-2017-147039	2017-06-29	11.54	11
12	AA-10375	Allen Arnold	CA-2014-158064	2014-04-21	16.52	1
13	AA-10375	Allen Arnold	CA-2014-130729	2014-10-24	34.27	2
14	AA-10375	Allen Arnold	CA-2015-140921	2015-02-03	28.40	3
15	AA-10375	Allen Arnold	CA-2015-140921	2015-02-03	149.97	4
16	AA-10375	Allen Arnold	CA-2015-109939	2015-05-08	5.25	5
17	AA-10375	Allen Arnold	CA-2015-114503	2015-11-13	84.96	6
18	AA-10375	Allen Arnold	CA-2016-126613	2016-07-10	16.77	7
19	AA-10375	Allen Arnold	CA-2016-131065	2016-11-14	499.98	8
20	AA-10375	Allen Arnold	CA-2016-131065	2016-11-14	5.28	9
21	AA-10375	Allen Arnold	CA-2016-131065	2016-11-14	8.26	10
22	AA-10375	Allen Arnold	US-2017-1694...	2017-09-07	16.90	11
23	AA-10375	Allen Arnold	US-2017-1694...	2017-09-07	39.96	12
24	AA-10375	Allen Arnold	CA-2017-100230	2017-12-11	14.95	13
25	AA-10375	Allen Arnold	CA-2017-100230	2017-12-11	17.94	14
26	AA-10375	Allen Arnold	CA-2017-100230	2017-12-11	116.98	15
27	AA-10480	Andrew Allen	CA-2014-155271	2014-05-04	27.46	1

Query executed successfully. localhost\SQLEXPRESS (17.0 ... ARATI\HomePC (54) SuperstoreDB 00:00:00 Row: 1, Col: 1 9,994 rows

7. Top 3 customers based on total sales (Window Function)

```
WITH customer_sales AS (  
    SELECT c.customer_id, c.customer_name, SUM(o.sales) AS total_sales  
    FROM orders o JOIN customers c ON o.customer_id = c.customer_id  
    GROUP BY c.customer_id, c.customer_name  
)  
  
ranked AS (  
    SELECT customer_id, customer_name, ROUND(total_sales, 2) AS total_sales,  
           RANK() OVER (ORDER BY total_sales DESC) AS sales_rank  
    FROM customer_sales  
)  
  
SELECT * FROM ranked WHERE sales_rank <= 3;
```

```
227  
228 --7. Top 3 customers based on total sales (Window Function)  
229 WITH customer_sales AS (  
230     SELECT c.customer_id, c.customer_name, SUM(o.sales) AS total_sales  
231     FROM orders o JOIN customers c ON o.customer_id = c.customer_id  
232     GROUP BY c.customer_id, c.customer_name  
233 ),  
234 ranked AS (  
235     SELECT customer_id, customer_name, ROUND(total_sales, 2) AS total_sales,  
236           RANK() OVER (ORDER BY total_sales DESC) AS sales_rank  
237     FROM customer_sales  
238 )  
239 SELECT * FROM ranked WHERE sales_rank <= 3;  
240  
241 /* =====  
242 STEP 6: Business Queries
```

100 % 28 0

Results Messages

	customer_id	customer_name	total_sales	sales_rank
1	SM-20320	Sean Miller	25043.07	1
2	TC-20980	Tamara Chand	19052.22	2
3	RB-19360	Raymond Buch	15117.35	3

STEP 3 :Final Combined Query (Customer Name, Total Sales, Rank)

```
WITH customer_sales AS (  
    SELECT c.customer_id, c.customer_name, SUM(o.sales) AS total_sales  
    FROM orders o  
    JOIN customers c ON o.customer_id = c.customer_id  
    GROUP BY c.customer_id, c.customer_name  
)  
SELECT  
    customer_name,  
    ROUND(total_sales, 2) AS total_sales,  
    RANK() OVER (ORDER BY total_sales DESC) AS rank  
FROM customer_sales  
ORDER BY rank;  
Output :
```

The screenshot shows a SQL IDE with the following query:

```
-- Final Combined Query (Customer Name, Total Sales, Rank)  
WITH customer_sales AS (  
    SELECT c.customer_id, c.customer_name, SUM(o.sales) AS total_sales  
    FROM orders o  
    JOIN customers c ON o.customer_id = c.customer_id  
    GROUP BY c.customer_id, c.customer_name  
)  
SELECT  
    customer_name,  
    ROUND(total_sales, 2) AS total_sales,  
    RANK() OVER (ORDER BY total_sales DESC) AS rank  
FROM customer_sales
```

The results are displayed in a table with 4 columns: customer_name, total_sales, and rank. The table contains 19 rows of data, ordered by rank from 1 to 19.

	customer_name	total_sales	rank
1	Sean Miller	25043.07	1
2	Tamara Chand	19052.22	2
3	Raymond Buch	15117.35	3
4	Tom Ashbrook	14595.62	4
5	Adrian Barton	14473.57	5
6	Ken Lonsdale	14175.23	6
7	Sanjit Chand	14142.34	7
8	Hunter Lopez	12873.30	8
9	Sanjit Engle	12209.44	9
10	Christopher Conant	12129.08	10
11	Todd Sumrall	11891.75	11
12	Greg Tran	11820.12	12
13	Becky Martin	11789.64	13
14	Seth Vernon	11470.94	14
15	Caroline Jumper	11164.97	15
16	Clay Ludtke	10880.56	16
17	Maria Etezadi	10663.73	17
18	Karen Ferguson	10604.26	18
19	Bill Shonely	10501.65	19

Query executed successfully. localhost\SQLEXP

8. Mini Project: Customer Sales Insights

1. Top 5 customers by total sales

SQL Query:

```
WITH customer_sales AS (  
    SELECT c.customer_id, c.customer_name, SUM(o.sales) AS total_sales  
    FROM orders o JOIN customers c ON o.customer_id = c.customer_id  
    GROUP BY c.customer_id, c.customer_name  
)  
ranked AS (  
    SELECT customer_id, customer_name, ROUND(total_sales, 2) AS total_sales,  
           RANK() OVER (ORDER BY total_sales DESC) AS sales_rank  
    FROM customer_sales  
)  
SELECT * FROM ranked WHERE sales_rank <= 5;
```

Output :

```
258 -- Query 7: Top 5 customers by total sales  
259 WITH customer_sales AS (  
260     SELECT c.customer_id, c.customer_name, SUM(o.sales) AS total_sales  
261     FROM orders o JOIN customers c ON o.customer_id = c.customer_id  
262     GROUP BY c.customer_id, c.customer_name  
263 )  
264 ranked AS (  
265     SELECT customer_id, customer_name, ROUND(total_sales, 2) AS total_sales,  
266           RANK() OVER (ORDER BY total_sales DESC) AS sales_rank  
267     FROM customer_sales  
268 )  
269 SELECT * FROM ranked WHERE sales_rank <= 5;  
270
```

100 % 28 0

Results Messages

	customer_id	customer_name	total_sales	sales_rank
1	SM-20320	Sean Miller	25043.07	1
2	TC-20980	Tamara Chand	19052.22	2
3	RB-19360	Raymond Buch	15117.35	3
4	TA-21385	Tom Ashbrook	14595.62	4
5	AB-10105	Adrian Barton	14473.57	5

2. Bottom 5 customers by total sales (low-value customers)

SQL Query:

```
WITH customer_sales AS (  
    SELECT c.customer_id, c.customer_name, SUM(o.sales) AS total_sales  
    FROM orders o JOIN customers c ON o.customer_id = c.customer_id  
    GROUP BY c.customer_id, c.customer_name  
)  
ranked AS (  
    SELECT customer_id, customer_name, ROUND(total_sales, 2) AS total_sales,  
           RANK() OVER (ORDER BY total_sales ASC) AS low_rank  
    FROM customer_sales  
)  
SELECT * FROM ranked WHERE low_rank <= 5;
```

Output (executed result):

```
271  -- Query 8: Bottom 5 customers by total sales (low-value customers)  
272  WITH customer_sales AS (  
273      SELECT c.customer_id, c.customer_name, SUM(o.sales) AS total_sales  
274      FROM orders o JOIN customers c ON o.customer_id = c.customer_id  
275      GROUP BY c.customer_id, c.customer_name  
276  ),  
277  ranked AS (  
278      SELECT customer_id, customer_name, ROUND(total_sales, 2) AS total_sales,  
279             RANK() OVER (ORDER BY total_sales ASC) AS low_rank  
280      FROM customer_sales  
281  )  
282  SELECT * FROM ranked WHERE low_rank <= 5;  
283  
284  -- Query 9: Customers who placed only ONE order  
285  WITH order_counts AS (  
286      SELECT customer_id, COUNT(*) AS order_count  
287      FROM orders  
288      GROUP BY customer_id  
289      HAVING order_count = 1  
290  )  
291  SELECT c.customer_id, c.customer_name  
292  FROM customers c JOIN order_counts oc ON c.customer_id = oc.customer_id
```

100 %



28



0



Results

Messages

	customer_id	customer_name	total_sales	low_rank
1	TS-21085	Thais Sissman	4.84	1
2	LD-16855	Lela Donovan	5.30	2
3	CJ-11875	Carl Jackson	16.52	3
4	MG-18205	Mitch Gastineau	16.74	4
5	RS-19870	Roy Skaria	22.33	5

3. Customers who placed only ONE order

SQL Query:

```
WITH order_counts AS (  
    SELECT c.customer_id, c.customer_name,  
           COUNT(DISTINCT o.order_id) AS order_count,  
           SUM(o.sales) AS total_sales  
    FROM orders o JOIN customers c ON o.customer_id = c.customer_id  
    GROUP BY c.customer_id, c.customer_name  
)  
SELECT customer_id, customer_name, order_count, ROUND(total_sales, 2) AS total_sales  
FROM order_counts  
WHERE order_count = 1  
ORDER BY total_sales DESC;
```

Output :

```
283  
284 -- Query 9: Customers who placed only ONE order  
285 WITH order_counts AS (  
286     SELECT c.customer_id, c.customer_name,  
287            COUNT(DISTINCT o.order_id) AS order_count,  
288            SUM(o.sales) AS total_sales  
289     FROM orders o JOIN customers c ON o.customer_id = c.customer_id  
290     GROUP BY c.customer_id, c.customer_name  
291 )  
292 SELECT customer_id, customer_name, order_count, ROUND(total_sales, 2) AS total_sales  
293 FROM order_counts  
294 WHERE order_count = 1  
295 ORDER BY total_sales DESC;  
296  
297 -- Query 10: Top 3 customers per Region (PARTITION BY)
```

100 % 28 0

Results Messages

	customer_id	customer_name	order_count	total_sales
1	JC-15385	Jenna Caffey	1	1058.11
2	SM-20905	Susan MacKendrick	1	1043.05
3	TC-21145	Theresa Coyne	1	1038.26
4	JR-15700	Jocasta Rupert	1	863.88
5	PH-18790	Patricia Hirasaki	1	729.65
6	AO-10810	Anthony O'Donnell	1	161.28
7	RM-19750	Roland Murray	1	98.35
8	AR-10570	Anemone Ratner	1	88.15
9	RE-19405	Ricardo Emerson	1	48.36
10	MG-18205	Mitch Gastineau	1	16.74
11	CJ-11875	Carl Jackson	1	16.52
12	LD-16855	Lela Donovan	1	5.30

4. Customers with above-average sales

```
WITH customer_sales AS (  
    SELECT c.customer_id, c.customer_name, SUM(o.sales) AS total_sales  
    FROM orders o JOIN customers c ON o.customer_id = c.customer_id  
    GROUP BY c.customer_id, c.customer_name  
)  
SELECT customer_id, customer_name, ROUND(total_sales,2) AS total_sales  
FROM customer_sales  
WHERE total_sales > (SELECT AVG(total_sales) FROM customer_sales)  
ORDER BY total_sales DESC;
```

```
297 WITH customer_sales AS (  
298     SELECT c.customer_id, c.customer_name, SUM(o.sales) AS total_sales  
299     FROM orders o JOIN customers c ON o.customer_id = c.customer_id  
300     GROUP BY c.customer_id, c.customer_name  
301 )  
302 SELECT customer_id, customer_name, ROUND(total_sales,2) AS total_sales  
303 FROM customer_sales  
304 WHERE total_sales > (SELECT AVG(total_sales) FROM customer_sales)
```

100 % 28 0 ↑ ↓

Results Messages

	customer_id	customer_name	total_sales
1	SM-20320	Sean Miller	25043.07
2	TC-20980	Tamara Chand	19052.22
3	RB-19360	Raymond Buch	15117.35
4	TA-21385	Tom Ashbrook	14595.62
5	AB-10105	Adrian Barton	14473.57
6	KL-16645	Ken Lonsdale	14175.23
7	SC-20095	Sanjit Chand	14142.34
8	HL-15040	Hunter Lopez	12873.30
9	SE-20110	Sanjit Engle	12209.44
10	CC-12370	Christopher Conant	12129.08
11	TS-21370	Todd Sumrall	11891.75
12	GT-14710	Greg Tran	11820.12
13	BM-11140	Becky Martin	11789.64
14	SV-20365	Seth Vernon	11470.94
15	CJ-12010	Caroline Jumper	11164.97
16	CL-12565	Clay Ludtke	10880.56
17	ME-17320	Maria Etezadi	10663.73
18	KF-16285	Karen Ferguson	10604.26
19	BS-11365	Bill Shonely	10501.65

5. Highest order value per customer

```
SELECT o.customer_id, c.customer_name, o.order_id, o.sales AS highest_order_sales
```

```
FROM orders o
```

```
JOIN customers c ON o.customer_id = c.customer_id
```

```
WHERE o.sales = (SELECT MAX(o2.sales) FROM orders o2 WHERE o2.customer_id = o.customer_id)
```

```
ORDER BY o.sales DESC;
```

The screenshot shows a SQL Server Enterprise Manager window with a query executed successfully. The query is as follows:

```
SELECT o.customer_id, c.customer_name, o.order_id, o.sales AS highest_order_sales
FROM orders o
JOIN customers c ON o.customer_id = c.customer_id
WHERE o.sales = (SELECT MAX(o2.sales) FROM orders o2 WHERE o2.customer_id = o.customer_id)
ORDER BY o.sales DESC;
```

The results are displayed in a table with the following columns: customer_id, customer_name, order_id, and highest_order_sales. The table contains 30 rows of data, sorted by highest_order_sales in descending order.

customer_id	customer_name	order_id	highest_order_sales
SM-20320	Sean Miller	CA-2014-145317	22638.48
TC-20980	Tamara Chand	CA-2016-118689	17499.95
RB-19360	Raymond Buch	CA-2017-140151	13999.96
TA-21385	Tom Ashbrook	CA-2017-127180	11199.97
HL-15040	Hunter Lopez	CA-2017-166709	10499.97
AB-10105	Adrian Barton	CA-2016-117121	9892.74
SC-20095	Sanjit Chand	CA-2014-116904	9449.95
BS-11365	Bill Shonely	US-2016-107440	9099.93
SE-20110	Sanjit Engle	CA-2016-158841	8749.95
CC-12370	Christopher Conant	CA-2016-143714	8399.98
KL-16645	Ken Lonsdale	CA-2014-143917	8187.65
BM-11140	Becky Martin	CA-2014-139892	8159.95
GT-14635	Grant Thornton	US-2017-168116	7999.98
TB-21400	Tom Boeckenhauer	CA-2014-145541	6999.96
CM-12385	Christopher Martinez	CA-2015-145352	6354.95
AR-10540	Andy Reiter	CA-2017-138289	5443.96
DR-12940	Daniel Raglin	US-2016-140158	5399.91
TS-21370	Todd Sumrall	CA-2017-143112	5199.96
JW-15220	Jane Waco	CA-2017-135909	5083.96
EH-13765	Edward Hooks	CA-2016-136301	4912.59
KD-16270	Karen Daniels	US-2016-143819	4899.93
HM-14860	Harry Marie	US-2015-128587	4899.93
NC-18535	Nick Crebassa	CA-2017-149881	4799.98
JA-15970	Joseph Airdo	CA-2017-158379	4663.74
KD-16495	Keith Dawkins	CA-2015-114811	4643.80
JM-15865	John Murray	CA-2014-144624	4548.81
DP-13390	Dennis Pardue	CA-2016-138478	4535.98
CS-12505	Cindy Stewart	CA-2016-108196	4499.98
MJ-17740	Max Jones	CA-2016-100300	4476.80
TP-21415	Tom Prescott	CA-2017-118892	4416.17
OL-10765	Oliver Jones	CA-2015-117098	4404.00

Query executed successfully. localhost\SQLEXPRESS (17.0 ... ARATI\HomePC (54) SuperstoreDB 00:00:00 Row: 1, Col: 1 / 795 rows

9. Additional Insights

Along with the required tasks, I also implemented two additional SQL queries to practice more Window Function concepts such as PARTITION BY and NTILE.

1. Top 3 customers per Region (PARTITION BY)

SQL Query:

```
WITH customer_region_sales AS (  
    SELECT c.customer_id, c.customer_name, o.region, SUM(o.sales) AS total_sales  
    FROM orders o JOIN customers c ON o.customer_id = c.customer_id  
    GROUP BY c.customer_id, c.customer_name, o.region  
)  
ranked AS (  
    SELECT customer_id, customer_name, region, ROUND(total_sales, 2) AS total_sales,  
           RANK() OVER (PARTITION BY region ORDER BY total_sales DESC) AS rank_in_region  
    FROM customer_region_sales  
)  
SELECT * FROM ranked WHERE rank_in_region <= 3  
ORDER BY region, rank_in_region;
```

Output :

```
313 -- Query 10: Top 3 customers per Region (PARTITION BY)
314 WITH customer_region_sales AS (
315     SELECT c.customer_id, c.customer_name, o.region, SUM(o.sales) AS total_sales
316     FROM orders o JOIN customers c ON o.customer_id = c.customer_id
317     GROUP BY c.customer_id, c.customer_name, o.region
318 ),
319 ranked AS (
320     SELECT customer_id, customer_name, region, ROUND(total_sales, 2) AS total_sales,
321            RANK() OVER (PARTITION BY region ORDER BY total_sales DESC) AS rank_in_region
322     FROM customer_region_sales
323 )
324 SELECT * FROM ranked WHERE rank_in_region <= 3
325 ORDER BY region, rank_in_region;
326
327 -- Query 11: Customer segmentation into quartiles (NTILE)
328 WITH customer_sales AS (
329     SELECT c.customer_id, c.customer_name, SUM(o.sales) AS total_sales
```

100 % 28 0

	customer_id	customer_name	region	total_sales	rank_in_region
1	TC-20980	Tamara Chand	Central	18437.14	1
2	AB-10105	Adrian Barton	Central	12181.60	2
3	BM-11140	Becky Martin	Central	10539.90	3
4	TA-21385	Tom Ashbrook	East	13723.50	1
5	HL-15040	Hunter Lopez	East	10522.55	2
6	BS-11365	Bill Shonely	East	10022.29	3
7	SM-20320	Sean Miller	South	23669.21	1
8	SE-20110	Sanjit Engle	South	8805.04	2
9	GT-14635	Grant Thornton	South	8167.42	3
10	RB-19360	Raymond Buch	West	14345.28	1
11	KL-16645	Ken Lonsdale	West	8472.39	2

2.Customer segmentation into quartiles (NTILE)

SQL Query:

```
WITH customer_sales AS (  
    SELECT c.customer_id, c.customer_name, SUM(o.sales) AS total_sales  
    FROM orders o JOIN customers c ON o.customer_id = c.customer_id  
    GROUP BY c.customer_id, c.customer_name  
)  
segmented AS (  
    SELECT customer_id, customer_name, ROUND(total_sales, 2) AS total_sales,  
           NTILE(4) OVER (ORDER BY total_sales DESC) AS quartile  
    FROM customer_sales  
)  
SELECT quartile,  
       COUNT(*) AS num_customers,  
       ROUND(AVG(total_sales), 2) AS avg_sales_in_quartile,  
       ROUND(MIN(total_sales), 2) AS min_sales,  
       ROUND(MAX(total_sales), 2) AS max_sales  
FROM segmented  
GROUP BY quartile  
ORDER BY quartile;
```

Output :

```
327 -- Query 11: Customer segmentation into quartiles (NTILE)  
328 WITH customer_sales AS (  
329     SELECT c.customer_id, c.customer_name, SUM(o.sales) AS total_sales  
330     FROM orders o JOIN customers c ON o.customer_id = c.customer_id  
331     GROUP BY c.customer_id, c.customer_name  
332 ),  
333 segmented AS (  
334     SELECT customer_id, customer_name, ROUND(total_sales, 2) AS total_sales,  
335            NTILE(4) OVER (ORDER BY total_sales DESC) AS quartile  
336     FROM customer_sales  
337 )  
338 SELECT quartile,  
339        COUNT(*) AS num_customers,  
340        ROUND(AVG(total_sales), 2) AS avg_sales_in_quartile,  
341        ROUND(MIN(total_sales), 2) AS min_sales,  
342        ROUND(MAX(total_sales), 2) AS max_sales  
343 FROM segmented  
344 GROUP BY quartile  
345 ORDER BY quartile;  
346  
347 -- Query 12: Best-selling product per category (window function)  
348 WITH product_sales AS (  
349     SELECT product_id, category_id, SUM(sales) AS total_sales  
350     FROM orders o JOIN products p ON o.product_id = p.product_id  
351     GROUP BY product_id, category_id  
352 )  
353 segmented AS (  
354     SELECT product_id, category_id, ROUND(total_sales, 2) AS total_sales,  
355            NTILE(4) OVER (ORDER BY total_sales DESC) AS quartile  
356     FROM product_sales  
357 )  
358 SELECT quartile,  
359        COUNT(*) AS num_products,  
360        ROUND(AVG(total_sales), 2) AS avg_sales_in_quartile,  
361        ROUND(MIN(total_sales), 2) AS min_sales,  
362        ROUND(MAX(total_sales), 2) AS max_sales  
363 FROM segmented  
364 GROUP BY quartile  
365 ORDER BY quartile;
```

100 % 28 0 ↑ ↓

Results Messages

	quartile	num_customers	avg_sales_in_quartile	min_sales	max_sales
1	1	199	6364.070000	3785.28	25043.07
2	2	198	2903.460000	2256.39	3766.40
3	3	198	1669.570000	1146.05	2254.29
4	4	198	632.780000	4.84	1137.61

10. Observations

- Customer sales are unevenly distributed, with only a small percentage of customers contributing the highest revenue.
 - A few high-value orders significantly increase the overall average sales, while most orders have lower sales values.
 - Window functions such as RANK(), ROW_NUMBER(), and NTILE() made it easier to identify top-performing customers and group customers based on their sales.
 - Common Table Expressions (CTEs) improved query readability and simplified complex calculations involving customer sales.
 - Subqueries were useful for comparing customer sales against average sales and identifying customers with the highest order values.
 - Only a small number of customers placed a single order, indicating that most customers made repeat purchases.
 - Dividing the dataset into normalized tables (Customers, Products, and Orders) reduced data redundancy and made SQL queries more organized and efficient.
 - The additional business queries provided deeper insights into customer performance across different regions and sales segments.
-

11. Summary of Key Insights

- Sales are not evenly distributed across the dataset. Only about **23.6% of order records** and **37% of customers** have sales above the average, indicating that a relatively small number of high-value orders and customers contribute significantly to total sales.
- **Sean Miller** is the highest revenue-generating customer with total sales of **\$25,043.05**, and most of these sales come from the **South** region.
- Out of **793 customers**, **294 customers (approximately 37%)** have total sales above the overall customer average, making them the company's major revenue contributors.
- Only **12 customers (around 1.5%)** placed a single order. This indicates that most customers made repeat purchases, reflecting good customer retention.
- Customer segmentation using **NTILE(4)** shows that the top 25% of customers generate significantly higher sales than the bottom 25%, highlighting the importance of focusing on high-value customers.
- The **Technology** category, especially products such as **Copiers** and **Videoconferencing Units**, contributes to many of the highest-value sales in the dataset.

Conclusion

During this assignment, I learned how Subqueries, CTEs, and Window Functions can be used together to analyze sales data efficiently. Creating separate customer, product, and order tables also helped me understand database normalization. This assignment improved my SQL querying skills and gave me practical experience solving real business problems using SQL Server.